Distributions, latent variables and the VAE

Notation

x: observed output	z: a per-example latent draw	ε: fixed-distribution noise
θ: decoder / generator	φ: encoder	ψ: discriminator

p(z) is the prior. **pθ(x|z)** is the decoder likelihood. **pθ(z|x)** is the true posterior. **qφ(z|x)** is its learned approximation.

KL measures expected extra surprisal

```
KL(P ‖ Q) = Σᵢ Pᵢ log(Pᵢ/Qᵢ)
H(P,Q) = H(P) + KL(P ‖ Q)
```

Natural logs give nats. P weights the expectation. KL is nonnegative but asymmetric. A zero-weight term contributes zero; positive P against zero Q gives infinity.

For P=(0.8,0.2), Q=(0.6,0.4): KL(P‖Q)=0.091516222; reverse KL=0.104649629. A negative individual contribution is possible.

A model and a bound

```
pθ(x,z) = p(z) pθ(x|z)
pθ(x) = ∫ p(z) pθ(x|z) dz
ELBO = E_q[log pθ(x|z)] − KL(qφ(z|x) ‖ p(z))
log pθ(x) = ELBO + KL(qφ(z|x) ‖ pθ(z|x))
```

The loss has a **prior KL**. The gap has a **true-posterior KL**. Jensen's inequality uses concavity of log. An exact expectation is a lower bound; one Monte Carlo realisation need not be.

Reparameterize, then differentiate

```
std = exp(0.5 * logvar)
z = mu + std * epsilon       # epsilon ~ N(0,I)
KL = 0.5 * Σⱼ(muⱼ² + exp(logvarⱼ) − 1 − logvarⱼ)
```

For μ=1, σ=0.5, ε=−0.4 and f(z)=z²: z=0.8, f=0.64, ∂f/∂μ=1.6, ∂f/∂σ=−0.64, ∂f/∂logvar=−0.16. Keep ε fixed for finite differences.

Train and generate

For binary observations: use BCE with raw logits, sum over pixels; sum KL over latent coordinates; add per example, then average the batch. β=1 is the vanilla negative ELBO.

Generation: prior → z → decoder → observation distribution. The encoder is unnecessary. A probability image is different from a Bernoulli draw. A uniform latent grid is different from prior sampling.

Kingma & Welling · Auto-Encoding Variational Bayes

GAN updates and model comparison

One label convention

D(x) is the displayed probability that x is real. Code uses raw logits a(x). Real target: 1. Fake target for D: 0. Fake target for G: 1.

```
L_D = −E_real log D(x) − E_z log(1 − D(G(z)))
L_G = −E_z log D(G(z))        # non−saturating
```

At D(real)=0.8 and D(fake)=0.3: L_D=0.579818495 and L_G=1.203972804.

One complete alternating iteration

```
# D phase
D.requires_grad_(True)
opt_D.zero_grad(); opt_G.zero_grad()
fake = G(z)
loss_D = BCE_logits(D(real), ones)
loss_D += BCE_logits(D(fake.detach()), zeros)
loss_D.backward(); opt_D.step()

# G phase: fresh graph through updated D
opt_D.zero_grad(); opt_G.zero_grad()
D.requires_grad_(False)
fake = G(new_z)
loss_G = BCE_logits(D(fake), ones)
loss_G.backward(); opt_G.step()
D.requires_grad_(True)
```

Freezing D parameters preserves the input derivative. Detaching fake or using no_grad around D in the G phase removes the needed path. The minimal models have no BatchNorm/Dropout state.

Why the practical generator objective?

With D=sigmoid(a), the minimax derivative is −D; the non-saturating derivative is D−1. At D=0.01 they are −0.01 and −0.99. The complete parameter gradient also includes da/dx and dx/dθ.

Compare on the same questions

Model	Training	Generation / likelihood
Autoregressive	Next-token NLL	Sequential samples; exact factorised likelihood
VAE	Negative ELBO	Prior and decoder; marginal usually intractable
GAN	Adversarial objectives	G only; implicit sample distribution

What a loss curve cannot tell you

Inspect fixed-seed samples, diversity and coverage. Attractive samples may miss a mode. D≈1/2 is not proof of matching distributions. Generated scientific images and signals are not measurements. The supplied glyph and 2D runs are controlled teaching examples.

Goodfellow et al. · Generative Adversarial Nets · Lecture notes and worked examples